

LKP Project Report

Felix Burkard

Jakub Kliszewski

Georg Thiesen

July 29, 2026

1 Implementation Overview

The project extends OuicheFS by replacing the block index with extent-based addressing. Additional features are contiguous allocation, block reservation, sparse file support, defragmentation, and runtime statistics.

1.1 Implementation Status

Feature	Status
Extent-based index block	complete
Debugging ioctl	complete
Contiguous block allocator	complete
Write-time block reservation	complete
Garbage collector	complete
Sysfs statistics	complete
Supporting sparse files and holes	complete
Automatic defragmentation	complete
Advanced block allocator	not implemented

1.2 Read/Write

The core of the implementation consists of the `ouichefs_read()` and `ouichefs_write()` functions. The read function iterates block by block over the requested range until either the end of the user buffer or the end of the file is reached. When a hole is encountered the read function emits the correct number of zeros. The write function is divided into three parts. First, it creates a hole at the end of the file, when data is written beyond the current end of the file. Second, it allocates blocks to fill holes the write-loop would try to write into. Finally, the write-loop copies the data from the user buffer to the allocated blocks.

1.3 Allocation Strategy

A major design goal of the project was to reduce fragmentation and maximize extent sizes. Therefore, `ouichefs_alloc_contiguous()` searches the free block bitmap for consecutive free regions. The allocator scans the bitmap and selects the largest available contiguous run, limited by the number of blocks requested. While scanning, potential candidate runs are immediately marked as allocated to prevent concurrent writers from using the same blocks. To further reduce fragmentation a reservation mechanism was implemented. Instead of allocating blocks individually for every write operation, each file reserves a window of contiguous blocks. Reserved blocks are marked as used in the bitmap but are not immediately added to the extent list. Writes first consume these reserved blocks, new blocks are only allocated when they have been fully consumed. When allocating new blocks the allocator immediately reserves a new contiguous window rather than allocating only the blocks required to satisfy the current write. If allocation

fails because the disk ran out of blocks a garbage collector releases unused reservations from open files and retries the allocation.

1.4 Defragmentation

Since extent-based storage is sensitive to fragmentation, a defragmentation mechanism was implemented. The defragmentation process attempts to allocate a new contiguous region large enough to store the file, copies as much data as possible into this region, updates the extent list, and releases the old blocks. In case the defragmentation fails the old file state is restored. With every write operation the file system is checked for fragmentation. If the previously configured fragmentation threshold is reached defragmentation is triggered. To reduce the overhead of this check, counters for the number of regular files and the total number of extents are maintained in the superblock.

1.5 sysfs Statistics

The remaining statistics are computed dynamically when they are read from sysfs by scanning the inode table and the extent lists. This approach avoids unnecessary maintenance of additional counters that could become inconsistent after complex operations. Although this introduces additional computation when accessing the statistics, these operations are not part of the normal file operations but for debugging purposes and therefore have limited performance impact. The implementation also includes the calculation of total data blocks for testing purposes. The interface is integrated into the filesystem through the `ouichefs_sysfs` structure, which contains a kobject representing the sysfs directory and a pointer to the corresponding super block. A pointer to this structure is stored in `ouichefs_sb_info`. Thus, each mounted filesystem instance receives its own directory under sysfs. Each statistic is implemented as a callback function that is registered through a generic attribute structure, allowing all sysfs entries to share the same show and store handlers. This design minimizes duplicated code and simplifies the addition of new statistics.

2 Testing

The following tests are provided:

- Generic read/write tests
 - Simple write + read
 - Append
 - Overwrite
- File with Holes
 - Correct filesize
 - Empty blocks are zero
 - Data is accessible
 - Writing into the hole correct filesize check
 - Written data inside hole is correct
 - Original data is still correct
 - Bytes around written data remain zero
- Test get extents IOCTL

- Test extents edge cases
 - Test truncate file
 - Test truncate to size zero
 - Test extents big file size
- Test validate fewer extents with block allocator
- Test validate block reservation
- Test reservation GC
- Test sysfs
- Test sysfs on other partition
- Defragmentation test
 - Simple defragmentation test
 - Defrag preserves holes test
 - Partial defragmentation under low space
 - Automatic defragmentation
- Persistence Test

To run the tests the file `extent.sh` has to be executed inside the VM. The test script handles installing the module and mounting the image.

3 Conclusion

The tests show that the implemented extent-based allocation strategy successfully improves the management of file storage compared to a simple block-based approach. By grouping consecutive blocks into extents, the filesystem reduces fragmentation. The reservation mechanism further improves allocation efficiency by allowing future writes to use already reserved contiguous blocks, while the defragmentation process is able to reorganize fragmented files. However, some limitations remain, such as the cost of scanning the filesystem for statistics.